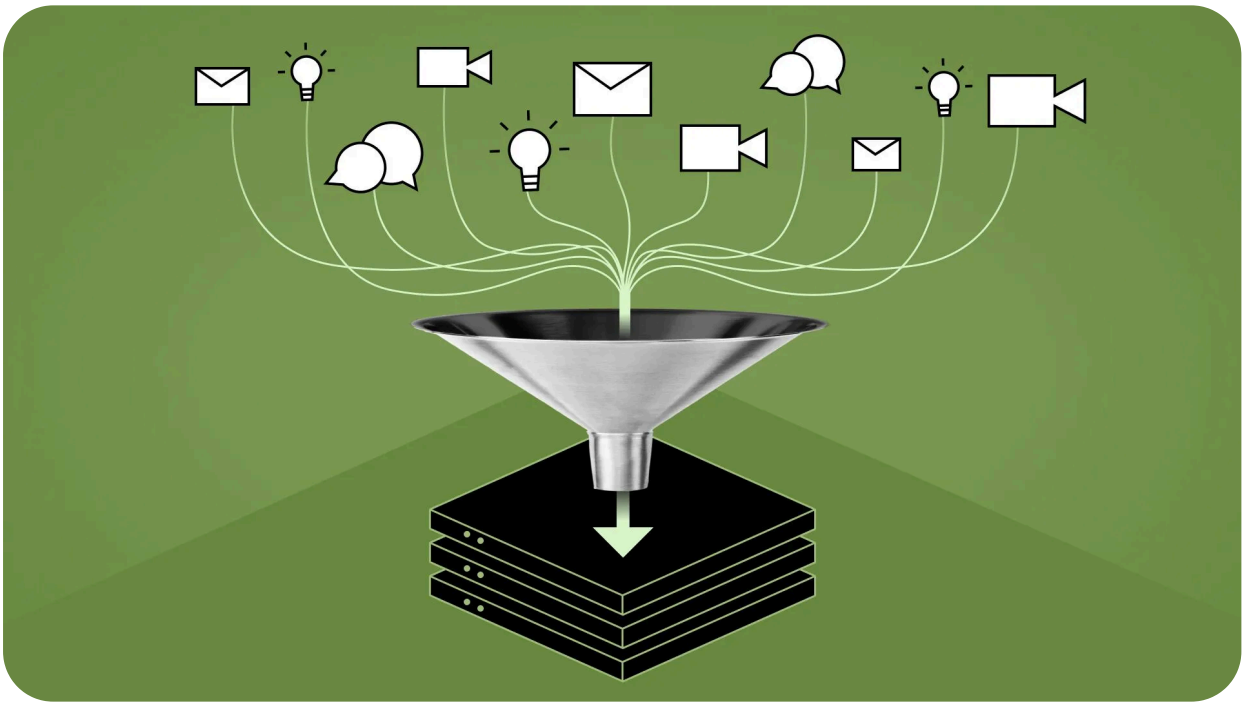
foundation capital

AI's trillion-dollar opportunity: Context graphs
AI 的万亿美元机遇：上下文图谱

12.22.2025 | By: Jaya Gupta, Ashu Garg

2025 年 12 月 22 日 | 作者：Jaya Gupta，Ashu Garg



The last generation of enterprise software created a trillion-dollar ecosystem by becoming systems of record. Salesforce for customers. Workday for employees. SAP for operations. Own the canonical data, own the workflow,

own the lock-in.

上一代企业软件通过成为记录系统，创造了一个万亿美元的生态系统。Salesforce 用于客户管理，Workday 用于员工管理，SAP 用于运营。掌控权威数据，掌控工作流程，掌控用户锁定。

The debate right now is whether those systems survive the shift to agents. Jamin Ball's recent post "Long Live Systems of Record" hit a nerve. Pushing back on the "agents kill everything" narrative, he argues that agents don't replace systems of record, they raise the bar for what a good one looks like.

目前的争论是，这些系统能否在向代理（agents）转型中生存下来。Jamin Ball 最近的一篇博客"记录系统万岁"引发了广泛讨论。他反驳了"代理会消灭一切"的说法，认为代理不会取代记录系统，而是提高了优秀记录系统应有的标准。

We agree. Agents are cross-system and action-oriented. The UX of work is separating from the underlying data plane. Agents become the interface, but something still has to be canonical underneath.

我们同意。代理人是跨系统的，且以行动为导向。工作的用户体验正在与底层数据平面分离。代理人将成为接口，但底层仍需要某种标准形式。

Where we go further is this: Ball's framing assumes the data agents need already lives somewhere, and agents just need better access to it plus better governance, semantic contracts, and explicit rules about which definition wins for which purpose.

我们进一步探讨的是：Ball 的框架假设所需数据已存在于某处，而数据代理只需获得更好的访问途径，以及更好的治理、语义契约，以及关于何种定义在何种用途下占优的明确规则。

That's half the picture. The other half is the missing layer that actually runs enterprises: **the decision traces** – the exceptions, overrides, precedents, and cross-system context that currently live in Slack threads, deal desk conversations, escalation calls, and people's heads.

这只是图的一半。另一半是目前缺失的、真正驱动企业运作的层面：决策轨迹——包括例外情况、覆盖操作、先例以及跨系统上下文，这些目前都存在于 Slack 线程、交易桌对话、升级电话以及人们的大脑中。

This is the distinction that matters:

这是至关重要的区别：

Rules tell an agent *what should happen in general* ("use official ARR for reporting")

规则告诉代理在一般情况下应该发生什么（"报告时使用官方 ARR"）

Decision traces capture *what happened in this specific case* ("we used X definition, under policy v3.2, with a VP exception, based on precedent Z, and here's what we changed").

决策轨迹记录了在此特定情况下发生了什么（"我们使用了 X 定义，依据 v3.2 政策，并通过 VP 例外，基于先例 Z，以下是我们的变更"）。

Agents don't just need rules. They need access to the decision traces that show how rules were applied in the past, where exceptions were granted, how conflicts were resolved, who approved what, and which precedents actually govern reality.

代理不仅需要规则。它们还需要访问决策轨迹，这些轨迹展示了规则过去是如何应用的、哪些例外被批准、冲突如何解决、谁批准了什么、以及哪些先例实际上支配着现实。

This is where systems of agents startups have a structural advantage. They sit **in the execution path**. They see the full context at decision time: what inputs were gathered across systems, what policy was evaluated, what exception route was invoked, who approved, and what state was written. If you persist those traces, you get something that doesn't exist in most enterprises today: a queryable record of how decisions were made.

这就是代理系统创业公司在结构上具有优势的地方。它们处于执行路径中。在决策时，它们能看到完整的上下文：跨系统收集了哪些输入、评估了哪种策略、调用了哪种例外路径、谁批准了、以及写入了什么状态。如果你持久化这些轨迹，就能获得今天大多数企业中不存在的东西：一个可查询的决策记录。

We call the accumulated structure formed by those traces a **context graph**: not "the model's chain-of-thought," but a living record of decision traces stitched across entities and time so precedent becomes searchable. Over time, that context graph becomes the real source of truth for autonomy –

because it explains not just *what* happened, but *why it was allowed* to happen.

我们称之为由这些轨迹形成的累积结构为“上下文图”：它并非“模型的思维链”，而是一个跨越实体和时间、将决策轨迹连接起来的动态记录，使得过往经验变得可搜索。随着时间推移，这个上下文图将成为自主性的真正事实来源——因为它不仅解释了发生了什么，还解释了为何允许其发生。

The core question isn't whether existing systems of record survive. It's whether entirely new ones emerge – systems of record for decisions, not just objects – and whether those become the next trillion-dollar platforms.

核心问题不在于现有记录系统能否生存，而在于全新的系统是否会出现——那些用于记录决策而非仅仅记录对象的系统，以及它们是否会成为下一个万亿美元级平台。

What systems of record don't capture
哪些记录系统无法捕捉

Agents are shipping into real workflows – contract review, quote-to-cash, support resolution – and teams are hitting a wall that governance alone can't solve.

代理正在进入实际工作流程——如合同审查、报价到收款、支持问题解决——而团队正面临一个仅靠治理无法解决的瓶颈。

The wall isn't missing data. It's missing decision traces. Agents run into the same ambiguity humans resolve every day with judgment and organizational memory. But the inputs to those judgments aren't stored as durable artifacts. Specifically:

墙并非缺少数据，而是缺少决策轨迹。代理系统遇到的歧义，正是人类每天通过判断力和组织记忆来解决的问题。但这些判断的输入信息并未被保存为持久化的记录。具体来说：

- **Exception logic that lives in people's heads.** “We always give healthcare companies an extra 10% because their procurement cycles are brutal.” That's not in the CRM. It's tribal knowledge passed down through onboarding and side conversations.

  存在于人们头脑中的例外逻辑。“我们总是给医疗公司额外10%的优惠，因为他们的采购周期非常严苛。”这一规则并未记录在CRM中，而是通过入职培训和闲聊传递下来的部落知识。

- **Precedent from past decisions.** "We structured a similar deal for Company X last quarter – we should be consistent." No system links those two deals or records why the structure was chosen.

  以往决策的先例。"我们上个季度为公司 X 设计了类似的交易结构——我们应该保持一致。"没有系统将这两个交易关联起来，也没有记录当初为何选择了这种结构。

- **Cross-system synthesis.** The support lead checks the customer's ARR in Salesforce, sees two open escalations in Zendesk, reads a Slack thread flagging churn risk, and decides to escalate. That synthesis happens in their head. The ticket just says "escalated to Tier 3."

  跨系统的综合判断。支持负责人在 Salesforce 中查看客户的年度经常性收入（ARR），发现 Zendesk 中有两个待处理的升级工单，并阅读了 Slack 群组中提示客户流失风险的讨论，最终决定升级处理。这个综合判断发生在他们的脑海中。而工单上仅记录"已升级至三级"。

- **Approval chains that happen outside systems.** A VP approves a discount on a Zoom call or in a Slack DM. The opportunity record shows the final price. It doesn't show who approved the deviation or why.

  在系统外发生的审批流程。一位副总裁在 Zoom 通话或 Slack 私信中批准了折扣。机会记录显示了最终价格，但没有显示谁批准了偏差以及为什么批准。

This is what "never captured" means. Not that the data is dirty or siloed, but that the reasoning connecting data to action was never treated as data in the first place.

这就是"从未被捕获"的含义。并不是数据不干净或孤立，而是将数据与行动联系起来的逻辑从未被当作数据处理过。

The context graph is the enduring layer
上下文图是持久的层

When startups instrument the agent orchestration layer to emit a decision trace on every run, they get something enterprises almost never have today: a structured, replayable history of how context turned into action.

当初创公司将代理编排层的工具化，以在每次运行时输出决策轨迹时，他们便获得了一种企业今天几乎从未拥有的东西：一种结构化的、可以回放的、展示上下文如何转化为行动的历史记录。

What does this look like in practice? A renewal agent proposes a 20% discount. Policy caps renewals at 10% unless a service-impact exception is approved. The agent pulls three SEV-1 incidents from PagerDuty, an open "cancel unless fixed" escalation in Zendesk, and the prior renewal thread where a VP approved a similar exception last quarter. It routes the exception to Finance. Finance approves. The CRM ends up with one fact: "20% discount."

在实际操作中会是什么样子呢？一位续保代理提出 20%的折扣。保单规定续保折扣上限为 10%，除非有服务影响的例外情况被批准。该代理从 PagerDuty 系统中调取了三个 SEV-1 级别事件，一个在 Zendesk 中开放的"除非修复否则取消"的升级工单，以及上季度一位副总裁批准过类似例外的先前续保线程。该代理将此例外情况路由至财务部门。财务部门批准了该申请。最后，CRM 系统中只保留了一条事实："20%折扣"。

Once you have decision records, the "why" becomes first-class data. Over time, these records naturally form a **context graph**: the entities the business already cares about (accounts, renewals, tickets, incidents, policies, approvers, agent runs) connected by decision events (the moments that matter) and "why" links. Companies can now audit and debug autonomy and turn exceptions into precedent instead of re-learning the same edge case in Slack every quarter.

一旦拥有决策记录，"为什么"就变成了第一类数据。随着时间的推移，这些记录会自然形成一个上下文图：业务 already 关注的实体（账户、续费、工单、事件、策略、审批人、代理运行）通过决策事件（关键时刻）和"为什么"链接相互连接。公司现在可以审计和调试自主性，并将异常情况转化为先例，而不是每季度在 Slack 中重新学习相同的边缘案例。

The feedback loop is what makes this compound. Captured decision traces become searchable precedent. And every automated decision adds another trace to the graph.

反馈循环使这一切变得复合。捕获的决策轨迹成为可搜索的先例。每一次自动化决策都会为图增加一条新的轨迹。

None of this requires full autonomy on day one. It starts with human-in-the-loop: the agent proposes, gathers context, routes approvals, and records the trace. Over time, as similar cases repeat, more of the path can be automated because the system has a structured library of prior decisions and exceptions. Even when a human still makes the call, the graph keeps growing, because the workflow layer captures the inputs, approval, and rationale as durable

precedent instead of letting it die in Slack.

这一切并不需要在第一天就实现完全的自主性。它始于“人在回路”：代理提出建议、收集上下文、路由审批并记录轨迹。随着时间的推移，随着类似情况的重复，更多路径可以被自动化，因为系统拥有结构化的先前决策和异常的库。即使人类仍然做出最终决定，图也会持续增长，因为工作流层将输入、审批和理由作为持久的先例进行捕获，而不是让它在 Slack 中消失。

Why incumbents can't build the context graph
为什么现有企业无法构建上下文图

Ball is optimistic that existing players evolve into this architecture. Warehouses become "truth registries," while CRMs become "state machines with APIs." His is a narrative of evolution, not replacement.

鲍尔乐观地认为，现有玩家将演进为这种架构。仓库将成为“真相注册表”，而 CRM 则会变成“带有 API 的状态机”。他描述的是一个演进的过程，而非替换。

That might work for making existing data more accessible. It doesn't work for capturing decision traces.

这种方法可能有助于让现有数据更加易于访问。但无法捕捉决策轨迹。

Operational incumbents are siloed and prioritize current state.

运营领域的现有企业是孤立的，并且优先考虑当前状态。

Salesforce is pushing Agentforce, ServiceNow has Now Assist, and Workday is building agents for HR. Their pitch is "we have the data, now we add the intelligence."

Salesforce 正在推出 Agentforce，ServiceNow 推出了 Now Assist，而 Workday 正在为人力资源构建代理人。他们的口号是“我们拥有数据，现在我们增加智能。”

But these agents inherit their parent's architectural limitations. Salesforce is built on current state storage: it knows what the opportunity looks like now, not what it looked like when the decision was made. When a discount gets approved, the context that justified it isn't preserved. You can't replay the state of the world at decision time, which means you can't audit the decision, learn

from it, or use it as precedent.

但这些代理继承了其父级的架构限制。Salesforce 基于当前状态存储：它知道机会当前的样子，而不知道决策时的样子。当折扣获得批准时，支持该决策的上下文并未被保留。你无法重放决策时刻的世界状态，这意味着你无法审计决策、从中学习或将其用作先例。

They also inherit their parent's blind spots. A support escalation doesn't live in Zendesk alone. It depends on customer tier from the CRM, SLA terms from billing, recent outages from PagerDuty, and the Slack thread flagging churn risk. No incumbent sees this because no incumbent sits in the cross-system path.

它们也继承了父级的盲点。支持升级并非仅存在于 Zendesk 中，它还依赖于来自 CRM 的客户等级、来自计费系统的 SLA 条款、来自 PagerDuty 的最近故障以及通过 Slack 线程标记的客户流失风险。没有现有玩家能看清这一点，因为没有一家现有玩家位于跨系统路径上。

The warehouse players have a different problem: they're in the read path, not the write path.

数据仓库的玩家面临的是不同的问题：他们在读取路径上，而非写入路径上。

Ball positions Snowflake and Databricks as the "truth registry" layer. Both are leaning in – Snowflake pushing Cortex and acquiring Streamlit, Databricks acquiring Neon and launching Lakebase and AgentBricks. The pitch: data platforms replace systems of record and become the foundation for AI agents.

Ball 将 Snowflake 和 Databricks 定位为"真相注册层"。两者都在积极布局——Snowflake 推动 Cortex 并收购 Streamlit，Databricks 则收购 Neon 并推出 Lakebase 和 AgentBricks。其主张是：数据平台将取代现行信息系统，成为 AI 代理的基础。

Warehouses do have a time-based view. You can query historical snapshots, track how metrics changed, and compare state across periods. But warehouses receive data via ETL after decisions are made. By the time data lands in Snowflake, the decision context is gone.

数据仓库确实具备基于时间的视图。你可以查询历史快照，追踪指标的变化，并在不同时期之间进行状态比较。但数据是通过 ETL 在决策做出后才进入仓库的。当数据抵达 Snowflake 时，决策的上下文已经丢失。

A system that only sees reads, after the fact, can't be the system of record for decision lineage. It can tell you what happened, but it can't tell you why.

仅事后查看读取操作的系统，无法成为决策血缘的权威系统。它能告诉你发生了什么，但无法告诉你为什么。

Databricks is further along in putting the pieces together. But being close to where agents get built isn't the same as being in the execution path where decisions happen.

Databricks 在整合这些组件方面已经走得更远。但接近智能体构建的位置，并不等于处于决策发生的工作流程路径上。

Systems of agents startups have a structural advantage: they're in the orchestration path.

智能体系统创业公司拥有结构性优势：它们位于编排路径上。

When an agent triages an escalation, responds to an incident, or decides on a discount, it pulls context from multiple systems, evaluates rules, resolves conflicts, and acts. The orchestration layer sees the full picture: what inputs were gathered, what policies applied, what exceptions were granted, and why. Because it's executing the workflow, it can capture that context at decision time – not after the fact via ETL, but in the moment, as a first-class record.

当智能体进行升级分类、响应事件或决定折扣时，它会从多个系统获取上下文、评估规则、解决冲突并采取行动。编排层能够看到完整的画面：收集了哪些输入、应用了哪些策略、批准了哪些例外，以及原因是什么。由于它在执行工作流程，因此可以在决策时刻捕捉这些上下文——不是通过 ETL 事后捕获，而是在实时作为第一类记录。

That's the context graph, and that will be the single most valuable asset for companies in the era of AI.

这就是上下文图，它将成为人工智能时代企业最具价值的单一资产。

Incumbents will fight back. They'll try acquisitions to bolt on orchestration capabilities. They'll lock down APIs and adopt egress fees to make data extraction expensive – the same playbook hyperscalers used. They'll build their own agent frameworks and push "keep everything in our ecosystem"

narratives.

现有企业将展开反击。他们会尝试通过收购来增加编排能力。他们会锁定 API 并采用出口费用，使数据提取变得昂贵——这与超大规模云服务商所采用的策略相同。他们将建立自己的代理框架，并推动“将所有内容保留在我们的生态系统中”的叙事。

But capturing decision traces requires being in the execution path at commit time, not bolting on governance after the fact. Incumbents can make extraction harder, but they can't insert themselves into an orchestration layer they were never part of.

但捕捉决策轨迹需要在提交时处于执行路径中，而不是事后附加治理。现有企业可以增加数据提取的难度，但他们无法插入到自己从未参与的编排层中。

Three paths for startups 初创企业的三条路径

Systems of agents startups will take different paths, each of which has its own tradeoffs.

代理系统初创企业将采取不同的发展路径，每条路径都有其自身的优势与权衡。

Some will replace existing system of records from day one. A CRM or ERP rebuilt around agentic execution, with event-sourced state and policy capture native to the architecture. This is hard because incumbents are entrenched, but it becomes viable at transition moments.

一些将从第一天起就取代现有的记录系统。一种围绕代理执行构建的 CRM 或 ERP，采用事件源化状态和原生的策略捕捉架构。这很难，因为现有企业已经根深蒂固，但在转型时刻，它变得可行。

Of the many startups going after the AI SDR category, Regie has chosen to build an AI-native sales engagement platform to replace legacy platforms like Outreach/Salesloft, which were designed for humans executing sequences across a fragmented toolchain. Regie is designed for a mixed team where the agent is a first-class actor: it can prospect, generate outreach, run follow-ups,

handle routing, and escalate to humans.

在众多进入 AI SDR 领域的初创公司中，Regie 选择构建一个原生 AI 销售参与平台，以取代 Outreach/Salesloft 等传统平台，这些平台最初是为人类在分散的工具链中执行序列而设计的。Regie 专为混合团队而设计，其中代理是第一类参与者：它可以进行潜在客户开发、生成联络信息、执行后续跟进、处理路由，并在必要时升级到人类处理。

Some will replace modules rather than entire systems. These startups target specific sub-workflows where exceptions and approvals concentrate, then become the system of record for those decisions while syncing final state back to the incumbent.

一些则将取代模块而非整个系统。这些初创公司针对特定的子工作流，这些工作流中异常和审批集中的地方，然后成为这些决策的记录系统，同时将最终状态同步回现有系统。

Maximor is doing this in finance: automating cash, close management, and core accounting workflows without ripping out the GL. The ERP remains the ledger, but Maximor becomes the source of truth where the reconciliation logic lives.

Maximor 正在金融领域实现这一目标：自动化现金流、结账管理和核心会计工作流，而无需替换总账。ERP 仍然是分类账，但 Maximor 成为真相来源，其中包含对账逻辑。

Some will create entirely new systems of record. These startups start as orchestration layers, but they persist what enterprises never systematically stored: the decision-making trace. Over time that replayable lineage becomes the authoritative artifact. The agent layer stops being "just automation" and becomes the place the business goes to answer "why did we do that?"

一些企业将创建全新的记录系统。这些初创公司最初作为编排层起步，但它们保存了企业从未系统性存储的内容：决策的完整记录。随着时间推移，这种可回放的 lineage（追溯路径）将成为权威的资产。代理层不再仅仅是"自动化"，而成为企业用来回答"我们为什么要这么做？"这一问题的场所。

PlayerZero exemplifies this pattern. Production engineering sits at the intersection of SRE, support, QA, and dev: a classic "glue function" where humans carry context that software doesn't capture. PlayerZero starts by automating L2/L3 support, but the real asset is the context graph it builds: a

living model of how code, config, infrastructure, and customer behavior interact in reality. That graph becomes the source of truth for "why did this break?" and "will this change break production?": questions no existing system can answer.

PlayerZero 正是这一模式的典范。生产工程处于 SRE、支持、质量保证（QA）和开发的交汇点：一个典型的"粘合功能"，人类在此携带了软件无法捕捉的上下文信息。PlayerZero 从自动化 L2/L3 支持开始，但其真正资产是所构建的上下文图谱：一个动态的模型，展示了代码、配置、基础设施和客户行为在现实中的相互作用。这个图谱成为回答"为什么出问题了？"和"此次变更是否会影响生产环境？"等关键问题的真相来源——而这些问题，目前的任何系统都无法解答。

As startups pursue these paths, observability for agents will become critical infrastructure. As decision traces accumulate and context graphs grow, enterprises will need to monitor, debug, and evaluate agent behavior at scale.

随着初创公司沿着这些方向发展，对代理的可观测性将变得至关重要。随着决策轨迹的积累以及上下文图谱的扩展，企业将需要在大规模下监控、调试和评估代理的行为。

Arize is building the observability layer for this new stack – giving teams visibility into how agents reason, where they fail, and how their decisions perform over time. Just as Datadog became essential infrastructure for monitoring applications, Arize is positioned to become essential infrastructure for monitoring and improving agent decision quality.

Arize 正在为这一新型架构构建可观测性层——帮助团队了解代理如何进行推理、在何处出现故障，以及其决策随时间推移的表现。正如 Datadog 成为监控应用的必备基础设施一样，Arize 正处于成为监控和提升代理决策质量的必备基础设施的有利位置。

Key signals for founders 创始人应注意的关键信号

The signals for where to build overlap, but they aren't identical.

这些信号在何处构建存在重叠，但并不完全相同。

Two signals apply to all three opportunities:

三个机会均适用两个信号：

High headcount. If a company has 50 people doing a workflow manually (routing tickets, triaging requests, or reconciling data between systems), that's a signal. The labor exists because the decision logic is too complex to automate with traditional tooling.

人员数量多。如果一家公司有 50 人手动完成某项工作流程（例如路由工单、分类请求或在不同系统之间核对数据），这是一个信号。这些人工劳动的存在，是因为决策逻辑过于复杂，无法通过传统工具实现自动化。

Exception-heavy decisions. Routine, deterministic workflows don't need decision lineage: the agent just executes. The interesting surfaces are where the logic is complex, where precedent matters, and where "it depends" is the honest answer. Think use cases like deal desks, underwriting, compliance reviews, and escalation management.

异常情况较多的决策。常规的、确定性的工作流程不需要决策溯源：代理只需执行。真正有趣的是那些逻辑复杂、 precedent（先例）重要的场景，以及"视情况而定"才是真实答案的地方。例如交易台、承保、合规审查和升级管理等应用场景。

One signal points specifically to new system of record opportunities:

一个信号特别指向新的记录系统机会：

Organizations that exist at the intersection of systems. RevOps exists because someone has to reconcile sales, finance, marketing, and customer success. DevOps exists because someone has to bridge development, IT, and support. Security Ops sits between IT, engineering, and compliance.

存在于系统交叉点的组织。销售运营（RevOps）的存在，是因为需要有人来协调销售、财务、市场和客户成功。开发运维（DevOps）的存在，是因为需要有人来连接开发、IT 和客服。安全运营（Security Ops）则处于 IT、工程和合规之间。

These "glue" functions are a tell. They emerge precisely because no single system of record owns the cross-functional workflow. The org chart creates a role to carry the context that software doesn't capture.

这些"粘合"功能是一个明显的信号。它们之所以出现，恰恰是因为没有一个单一的记录系统能够拥有跨职能工作流程的所有权。组织架构图创建了一个角色，以承载软件无法捕捉的上下文信息。

An agent that automates that role doesn't just run steps faster. It can persist the decisions, exceptions, and precedents the role was created to produce. That's the path to a new system of record: not by ripping out an incumbent, but by capturing a category of truth that only becomes visible once agents sit in the workflow.

一个能够自动化该角色的代理不仅运行步骤更快。它还可以持久化该角色所创建的决策、例外情况和先例。这就是通向新记录系统之路：不是通过取代现有系统，而是通过捕捉一类只有在代理进入工作流后才显现的真理。

Systems of record, reimagined
重新构想的记录系统

The question isn't whether systems of record survive – they will. The question is whether the next trillion-dollar platforms are built by adding AI to existing data, or by capturing the decision traces that make data actionable.

问题不在于记录系统是否会持续存在——它们会。问题在于，下一个价值万亿美元的平台，是通过向现有数据中添加人工智能来构建，还是通过捕捉使数据可操作的决策轨迹来构建。

We think it's the latter. And the startups building context graphs today are laying the foundation.

我们认为是后者。而目前构建上下文图的初创公司正在打下基础。

Published on 12.22.2025 发布于 2025 年 12 月 22 日
Written by Jaya Gupta and Ashu Garg

作者：Jaya Gupta 和 Ashu Garg

Related Stories 相关故事